

OIC-DESIGN-2026-044: Terminal-Anchored Reachability Analysis — proving and disproving routes to an outcome

Field	Value
Status	<code>[DRAFT]</code> — proposes a new capability. Promotes what began as a multi-flow option in <code>OIC-DESIGN-2026-042</code> into its own design focus, because it is the core of what makes the generation pipeline worth building.
Date	2026-06-19
Owner	D. Leece
Primary purpose	Given an asset and its terminal compromise, enumerate the credible routes an adversary could take to reach it from different entry points — and, just as importantly, rule routes out , explaining why a feared entry does <i>not</i> credibly reach the outcome. The output is a corrected threat picture for an asset owner, reviewed by a human (the owner or an advisor).
Relationship to other docs	Sits inside the GENERATION pipeline. Extends <code>OIC-DESIGN-2026-042</code> (elicitation, four entry archetypes) and inherits the artifact decisions of <code>OIC-DESIGN-2026-043</code> (STIX canonical, Mermaid view, firewall). Each credible route is a separate STIX bundle; the compression pipeline (<code>OIC-DESIGN-2026-002</code>) remains downstream and untouched.

How to read this. §1-§2 are the *why*: the cognitive failure this corrects, and why disproving a route is as valuable as proving one. §3 is the model (terminal-anchored, work-backward, attack-tree shaped). §4 is the three-verdict taxonomy — the heart of the doc. §5 is the conditional-negative discipline (the safety-critical part). §6 is convergence analysis. §7 is output/firewall. §8 is the human-review model. §9 is guardrails and the sprint backlog.

1. The use case: correcting a distorted threat picture

The asset owner from `OIC-DESIGN-2026-042` has a vague asset and a handful of half-formed worries. This capability addresses *what those worries actually are* and how distorted they tend to be.

People running organizations are **bombarded with cyber-threat news**, and it produces two simultaneous distortions:

- **Misplaced worry** — fixation on a vivid, newsworthy vector ("our guest WiFi," "hackers," the latest ransomware headline) that may not credibly reach the asset they care about.
- **Blind spots** — the unglamorous paths nobody writes scare stories about (an over-provisioned service account, a vendor's standing remote access, a misconfigured trust) that *do* reach the asset.

A tool that only ever answers "yes, and here's how" amplifies the first distortion: every worry gets validated with a plausible-looking flow. The corrective is a tool that reasons **both directions** — confirming credible routes the owner hadn't considered, *and* ruling out feared routes that don't credibly reach the outcome, while flagging what those feared routes actually threaten instead.

The owner's frame is decisive here and shapes every design choice: **asset owners care about protecting assets, not the purity of analytical models**. An unorthodox entry point that credibly reaches the terminal is worth surfacing precisely *because* it is unorthodox — that is the blind spot. And a feared entry that is shown not to reach the terminal is worth stating plainly, because it reallocates finite attention. The value of a generative aid is exactly this: it can present an input path the user didn't think of, and it can test routes the user fixates on — which is how an **all-hazards mindset** benefits the process.

2. Why disproving a route is as valuable as proving one

Affirmative-only threat modeling has a structural bias: it validates whatever it is pointed at. Add the ability to *rule out*, and three things improve at once.

- **It corrects misplaced worry.** "The guest WiFi doesn't credibly reach your patient records (under stated conditions); here's what does" moves attention from a low-credibility fear to a real path. The ruling-out is what makes the ruling-in *trustworthy* — a tool that can say no is more credible when it says yes.
- **It exposes blind spots by contrast.** Enumerating routes by entry forces the question "what about the entries you *didn't* name?" — which is where the unglamorous credible paths live.
- **It embodies all-hazards reasoning.** Proving and disproving routes to a fixed outcome is precisely the all-hazards posture: start from the consequence, consider every way to get there, and let credibility — not familiarity or news cycle — decide what gets attention.

The asymmetry that governs the whole design: a false *positive* (an affirmative route that isn't real) wastes attention. A false *negative* (ruling out a route that *is* real) actively removes a defense — it talks the owner out of a genuine risk. Therefore the bar for emitting a

negative verdict is higher than for a positive, and every negative must be **conditional and assumption-explicit**, never absolute (§5). The tool must be structurally incapable of saying "you are safe from X."

3. The model: terminal-anchored, work-backward, attack-tree shaped

This is **attack-tree thinking**. An attack tree fixes the goal at the root and enumerates the disjoint ways to reach it; the root is the terminal, the branches are alternative routes. This capability generates the top OR-decomposition of the goal — rendered as N *separate* flows rather than one nested tree, so each route is its own reviewable artifact.

This matches how adversaries actually plan: **they start with the intended outcome and work backward** to the options available for achieving it. So the generation inverts the workbench's current input order:

- **Fixed input:** the terminal — the asset's compromise ("AD takeover," "patient-data exfiltration," "plant safety shutdown"). This is OTC-DESIGN-2026-042 Q1 ("what are you protecting"), now the *anchor* rather than one input among several.
- **Enumerated output:** for each candidate entry, the credible route(s) backward from terminal to entry — or a reasoned verdict that no credible route exists.

The four entry archetypes from 2026-042 (phishing / remote access / physical intrusion / over-provisioned AI agent) become **seeds for diversity** rather than a single user choice. The tool considers each against the terminal. It also considers entries the user *named* (the feared vectors, e.g. "guest WiFi") and may surface credible entries the user did *not* name (blind spots).

4. The three-verdict taxonomy (the core)

For each candidate entry considered against the fixed terminal, the tool returns exactly one of three verdicts:

Verdict	Meaning	Artifact produced
Credible route	A plausible path from this entry reaches the terminal.	A STIX Attack Flow bundle (the affirmative case), viewable in Mermaid.
Reaches a different terminal	This entry doesn't credibly reach <i>this</i> asset, but it does threaten something else.	A note of what it <i>does</i> threaten, optionally seeding a separate flow for that other terminal.
No credible path under stated conditions	Under explicit assumptions, no credible route from this entry reaches the terminal.	A reasoned negative: the verdict, the assumptions it rests on, and the condition that would flip it. Not a bundle.

Notes on each:

Credible route is the affirmative case and is *bounded* — exhibiting one plausible route settles it. Diversity across the credible routes must be load-bearing (§9): routes should differ through the *approach*, not just the first node, or the owner gets five first-steps glued onto one identical body.

Reaches a different terminal is the honest middle. "Implausible" is rarely absolute — a feared vector usually does *something*, just not the thing the owner fixated on. Naming what it actually threatens is more useful and more honest than a flat "no," and it connects to the `adjacent_terminals` idea already in the compression methodology.

No credible path is the safety-critical verdict and is governed entirely by §5.

5. The conditional-negative discipline (safety-critical)

A confident-sounding negative is more dangerous than a confident-sounding positive (§2). Ruling a path *in* is a bounded claim (exhibit one route). Ruling a path *out* is an **unbounded** claim — it asserts that *no* credible route exists, which requires having considered the routes you didn't think of. This is exactly the kind of universal negative an LLM will assert fluently and wrongly. The discipline that makes negatives safe:

1. **Never absolute.** The verdict is "no credible path **under these stated conditions**," never "impossible" and never "you are safe from X." The tool must be structurally incapable of issuing an unconditional all-clear.
2. **Assumptions explicit.** Every negative states what it assumed to reach the conclusion. "*Guest WiFi doesn't credibly reach patient records, assuming the guest network is isolated from the clinical network and shares no authentication.*"

3. **State the flip condition.** Name the specific condition that would change the verdict, so the owner can check it against reality. *"If the guest network is not isolated, or a shared jump host exists, this becomes a credible route."* This turns the negative into an actionable prompt: the owner immediately sees the one thing to verify.
4. **It is observed-shape, not control prescription.** A negative states the *structural precondition* under which the route doesn't exist. It is **not** a `succeeds when` gate — it does not tell the owner what control to deploy. Reasoning about whether a route exists is the generation pipeline's job; curating which controls gate a route that *does* exist is the compression pipeline's job (§7).

The net effect: the tool can say "this doesn't reach your crown jewels *if* these conditions hold, and here's the condition that would change that" — which is genuinely useful and corrects misplaced worry — without ever issuing the false assurance that does harm.

6. Convergence analysis: the payoff of generating routes as a set

Because all credible routes end at the same terminal, **where they converge is where the defensive insight concentrates**. If four of five routes pass through "obtain domain credentials" before the terminal, that chokepoint is visible as a structural fact of the route set.

This is the "shared gate" insight from the compression methodology (one control, many cascades), but now visible *within a single asset's threat picture* rather than across cards. The selection surface (§8) should make convergence visible, because it is the analytical payoff of generating routes as a set rather than one at a time.

Firewall caveat: convergence is a *structural* observation (these routes share a node) and belongs to the generated artifacts. The *control implication* of a chokepoint ("therefore invest here") is `succeeds when` reasoning and belongs to the compression pipeline. The set may make convergence visible; it must not assert what to do about it. Showing that routes pinch together is observed shape; prescribing the pinch as a control is curation.

7. Output and firewall

- **Each credible route = its own STIX 2.1 Attack Flow bundle** (the original "separate flow documents" requirement), tagged with the shared terminal so the viewer can group the set and compute convergence across it. Per `OIC-DESIGN-2026-043`: STIX canonical, Mermaid view, no hand-built `.afb`.
- **Negative and different-terminal verdicts** are reasoned notes, not bundles.

- **The firewall holds, twice over.** Generated artifacts — credible routes *and* negative verdicts — carry **observed shape only**: actions, assets, edges, and (for negatives) the structural preconditions/flip-conditions. They never carry `succeeds when` control gates; that content is added only in the compression pipeline, by a human curator, on the far side of the `.afb`/STIX seam. The negatives are the subtle case: their assumptions are observed-shape (preconditions for a route's existence), not control prescriptions, and must be written that way.
-

8. Human review is the backstop

A human reviews **all** outputs — the user themselves or an advisor in the process. This is a deliberate part of the design, not an afterthought, and it is what makes the asymmetric risk of §2 tolerable.

- The human is the final arbiter of credibility. The tool *proposes* routes and verdicts; the reviewer disposes. An unorthodox credible route the tool surfaces is exactly the kind of thing a reviewer should expand on further (the asset owner cares about protecting the asset, not model orthodoxy).
- Given the false-negative asymmetry, the review model should make negatives *more* prominent for scrutiny, not less — surface each negative with its assumptions and flip-condition front-and-center, so the reviewer can sanity-check the one condition that would overturn it.
- Selection feeds forward: the routes the reviewer keeps are the candidates that proceed to deeper analysis / compression. Ruling-out is a first-class outcome of review, not a failure of generation.

No approval gate on negatives — and that is deliberate. It is tempting to require an analyst sign-off before a negative verdict is shown. We explicitly do not, because it would chase a false comfort. A sign-off implies a negative can be made *authoritative* if the right person blesses it — but every negative rests on assumptions about a system nobody has complete intelligence on, so it cannot be made authoritative, and shouldn't pretend to be. A gate would launder irreducible uncertainty into apparent certainty, which is precisely the false-assurance failure the conditional-negative discipline exists to prevent.

The flip condition is the safeguard. It does not claim the negative is correct; it names the specific fact whose truth determines whether it is correct, and hands that residual uncertainty back to the human in a form they can act on. This is mature risk management: not the elimination of uncertainty, but developing comfort operating with unknown unknowns and missing intelligence signals, with assumptions made explicit enough that one knows what would change one's mind. A flip condition is a living risk-register entry — a documented assumption with a built-in trigger for re-evaluation ("judged not credible

because the guest network is isolated; if that changes, revisit") — not a closed verdict. The tool's job is to surface and structure those assumptions, not to resolve them; resolution requires intelligence the tool does not have, which is exactly what a gate would have falsely implied it could supply.

9. Guardrails and sprint backlog

Hard rules:

- **Negatives are always conditional** (§5). The tool cannot emit an unconditional "safe from X."
- **Load-bearing diversity.** Credible routes must differ through their approach, not just the entry node. A batch whose routes collapse to one common body is a defect — measure mid-route overlap and flag collapse.
- **Proactive blind-spots yes, proactive all-clears no.** The tool may *volunteer* credible entries the user didn't name (affirmative blind-spots are safe to surface). It must **not** proactively rule out vectors the user didn't raise (speculative negatives multiply the false-"no" risk). Rule out what the user names or fears; volunteer only affirmative discoveries.
- **Not every entry reaches every terminal**, and that's fine. Emitting fewer credible routes (with honest negatives for the rest) is correct; padding to hit a target N reintroduces fabrication.
- **Firewall:** observed shape only in all artifacts (§7).

Backlog:

- **R1 — Terminal-anchored generation mode (code).** Invert the generator to take the terminal as the fixed anchor and enumerate routes backward per entry archetype. Depends on the de-linearization work (`depends_on` edges) and STIX-native output (`OIC-DESIGN-2026-043`).
- **R2 — Three-verdict output schema (schema).** Each candidate entry yields credible / different-terminal / no-credible-path, with the negative carrying `assumptions` and `flip_condition` fields.
- **R3 — Conditional-negative prompt discipline (prompt/eval).** Constrain the model to conditional negatives only; eval specifically for any unconditional "safe"/"impossible"/"cannot" language and for over-confident universal negatives. This is the highest-risk component — eval it hardest.
- **R4 — Diversity metric (code/eval).** Quantify mid-route overlap across a batch; flag collapse so shallow diversity is caught before review.

- **R5 — Convergence view (app).** Selection surface that groups the route set by shared terminal and highlights convergence nodes (structural only — no control prescription).
 - **R6 — Review surface (app).** Present all verdicts for human disposition; make negatives prominent (assumptions + flip-condition visible); capture keep/reject/expand decisions that feed forward to compression.
-

This capability turns the generation pipeline from "here are some attack flows" into "here is a corrected, all-hazards picture of how your asset could fall — including the feared paths that don't credibly get there, and the unorthodox ones that do." The proving and the disproving are equally the product. Negatives are always conditional, diversity is always load-bearing, control logic always stays in compression, and a human always reviews — because the asset owner cares about protecting the asset, and the tool's job is to widen and correct their field of view, not to validate it. There is no approval gate on negatives: the flip condition is the safeguard, because mature risk management is not the elimination of uncertainty but operating with it explicitly — knowing the one fact that would change the answer is worth more than a stamp that pretends the answer is settled.